

INT202 Complexity of Algorithms
Tutorial 1

1. Find the asymptotic notation in Big-Oh

- (a) `int i = 1;
i++;
int m = i * i;` Time complexity is $O(1)$.
- (b) `int i = 1;
while (i < n) {
i = i * 2;
}` when $i=1, i'=2, k=1$
 $i=2, i'=4, k=2$
 $i=4, i'=8, k=3$.
therefore, $i=2^k, 2^k \leq n \rightarrow k = \log_2 n$ $\therefore$ Time complexity is $O(\log n)$.
- (c) `int i = 2;
while (i < n) {
i = i * i;
}` when $i=2, i'=2^2, k=1$
 $i=2^2, i'=2^4, k=2$
 $i=16, i'=2^8, k=3$
therefore, $i=2^{2^k} \rightarrow 2^k = \log_2 n \rightarrow k = \log_2 \log_2 n$ $\therefore$ Time complexity is $O(\log \log n)$.
- (d) `int j = 0;
for (int i = 0; i <= n; i++) {
j = i;
j = j * j;
}` The code within the for loop will not affect the loop itself, therefore, the highest order item will be the loop, time complexity can be determined by the for loop. $\therefore$ Time complexity is $O(n)$.
- (e) `for (int m = 0; m <= n; m++) {
i = 1;
while (i < n) {
i = i * 2;
}
}` The inner Time complexity is $O(\log n)$.
the outer loop is $O(n)$
 $\therefore$ Overall time complexity is $O(n \log n)$.

2. Multiplying two $n \times n$ square matrices $A = (a_{ij})_{1 \leq i, j \leq n}$ and $B = (b_{ij})_{1 \leq i, j \leq n}$ gives a matrix,

$$C = \begin{pmatrix} c_{11} & c_{12} & \dots & c_{1n} \\ c_{21} & c_{22} & \dots & c_{2n} \\ \dots & \dots & \dots & \dots \\ c_{n1} & c_{n2} & \dots & c_{nn} \end{pmatrix}$$

wherein
$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad \forall 1 \leq i, j \leq n$$

The following pseudo-code multiplies two square matrices A and B to get matrix C:

```
for i from 1 to n      % loop i
  for j from 1 to n    % loop j
    C[i][j] = 0;
    for k from 1 to n  % loop k
      C[i][j] = C[i][j] + A[i][k] * B[k][j];
    end for
  end for
end for
```

n^2
 n^3

Please give its asymptotic notation $T(n) \in O(g(n))$ (i.e $T(n)$ is Big-Oh of $g(n)$).

1. Outermost loop (i from 1 to n), run n times.
2. Middle loop (j from 1 to n), run n times for each i .
3. Innermost loop (k from 1 to n), run n times for each pair of i and j .

Total number of operations is $n \times n \times n = n^3$

The initialization adds n^2 ops, but dominated by $n^3 \therefore O(n^3)$